

Projeto I

Esta atividade foi desenvolvida como parte da avaliação da disciplina de Programação Web do Curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas do Instituto Federal de São Paulo, Campus Boituva. Se houver dúvidas sobre o enunciado, entre em contato através do e-mail: anisio.silva@ifsp.edu.br

1 Objetivo

Desenvolver uma **API REST** para gestão de uma **concessionária de veículos** com arquitetura MVC.

O sistema deve permitir o gerenciamento completo do cadastro de clientes, vendedores e carros, o controle de estoque e a emissão de notas fiscais de venda, seguindo rigorosamente as regras de negócio estabelecidas.

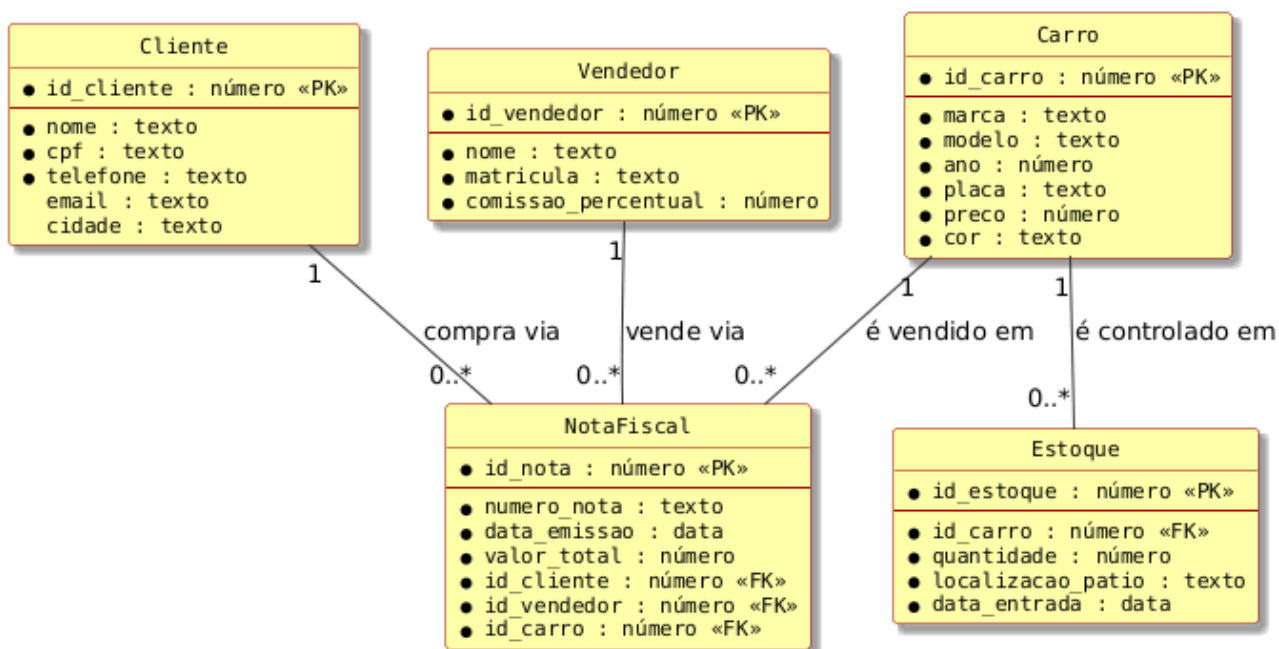


Figura 1: Diagrama de Entidade-Relacionamento do sistema

2 Regras de Negócio Detalhadas

As regras de negócio a seguir regulam o funcionamento do sistema. Todas as validações devem ser implementadas na camada de **serviço**, retornando respostas HTTP adequadas em caso de violação.

RN01 — Cadastro de Clientes

- O campo `cpf` é obrigatório e deve ser único no sistema. Não é permitido cadastrar dois clientes com o mesmo CPF.
- Os campos `nome`, `cpf` e `telefone` são obrigatórios. Os campos `email` e `cidade` são opcionais.
- Não é permitido remover um cliente que possua notas fiscais vinculadas a ele.

RN02 — Cadastro de Vendedores

- O campo `matricula` é obrigatório e deve ser único no sistema.
- O campo `comissao_percentual` deve ser um número positivo entre **0** e **30** (por cento).
- Não é permitido remover um vendedor que possua notas fiscais vinculadas a ele.

RN03 — Cadastro de Carros

- O campo `placa` é obrigatório e deve ser único no sistema. Não é permitido cadastrar dois carros com a mesma placa.
- O campo `ano` deve ser um número inteiro entre **1950** e o ano atual mais 1 (admitindo carros do próximo ano já em estoque).
- O campo `preco` deve ser um valor positivo maior que zero.
- Não é permitido remover um carro que possua registros em estoque ou notas fiscais vinculadas.

RN04 — Controle de Estoque

- Cada registro de estoque está vinculado a exatamente **um** carro (via `id_carro`). O carro referenciado deve existir no sistema.
- O campo `quantidade` deve ser um inteiro maior ou igual a **zero**. Uma quantidade igual a zero significa que o item está indisponível mas ainda consta no histórico.
- O campo `data_entrada` não pode ser uma data futura em relação à data atual do servidor.
- Não pode existir mais de um registro de estoque ativo para o mesmo `id_carro`. Para atualizar a quantidade, deve-se usar o endpoint de atualização (PUT).

RN05 — Emissão de Nota Fiscal

- Uma nota fiscal somente pode ser emitida se o carro associado possuir **quantidade** > 0 em estoque. Ao emitir, a quantidade em estoque é automaticamente decrementada em **1** unidade.
- O campo `numero_nota` é obrigatório e deve ser único no sistema.
- O campo `valor_total` deve ser positivo e maior que zero.
- Os campos `id_cliente`, `id_vendedor` e `id_carro` são obrigatórios e devem referenciar registros existentes no sistema.
- A `data_emissao` não pode ser uma data futura em relação à data atual do servidor.
- Notas fiscais **não podem ser removidas** após a emissão, garantindo integridade histórica.

RN06 — Consultas e Listagens

- Deve ser possível listar todas as notas fiscais de um cliente específico.
- Deve ser possível listar todas as notas fiscais de um vendedor específico.
- Deve ser possível consultar o estoque atual de um carro pelo seu id.
- Deve ser possível listar todos os carros com estoque disponível (quantidade > 0).

3 Documentação da API

A API segue os princípios REST, utilizando os verbos HTTP GET, POST, PUT e DELETE. Todas as requisições e respostas utilizam o formato **JSON**. A URL base sugerida é `http://localhost:3000`.

3.1 Clientes — /clientes

Método	Rota	Descrição
GET	/clientes	Lista todos os clientes cadastrados.
GET	/clientes/:id	Retorna os dados de um cliente pelo id.
POST	/clientes	Cadastra um novo cliente.
PUT	/clientes/:id	Atualiza os dados de um cliente existente.
DELETE	/clientes/:id	Remove um cliente (somente se não possuir notas fiscais).
GET	/clientes/notas/:id	Lista todas as notas fiscais de um cliente.

Corpo da requisição — POST / PUT /clientes

```
{
  "nome":      "João da Silva",      // obrigatório
  "cpf":       "123.456.789-00",     // obrigatório, único
  "telefone":  "(11) 99999-0000",    // obrigatório
  "email":     "joao@email.com",     // opcional
  "cidade":    "São Paulo"           // opcional
}
```

3.2 Vendedores — /vendedores

Método	Rota	Descrição
GET	/vendedores	Lista todos os vendedores.
GET	/vendedores/:id	Retorna um vendedor pelo id.
POST	/vendedores	Cadastra um novo vendedor.
PUT	/vendedores/:id	Atualiza um vendedor existente.
DELETE	/vendedores/:id	Remove um vendedor (sem notas vinculadas).
GET	/vendedores/notas/:id	Lista todas as notas fiscais de um vendedor.

Corpo da requisição — POST / PUT /vendedores

```
{
    "nome": "Ana Souza", // obrigatório
    "matricula": "VND-001", // obrigatório, único
    "comissao_percentual": 5.5 // obrigatório, entre 0 e 30
}
```

3.3 Carros — /carros

Método	Rota	Descrição
GET	/carros	Lista todos os carros.
GET	/carros/:id	Retorna um carro pelo id.
GET	/carros/disponiveis	Lista carros com estoque > 0.
POST	/carros	Cadastra um novo carro.
PUT	/carros/:id	Atualiza um carro existente.
DELETE	/carros/:id	Remove um carro (sem estoque ou notas).

Corpo da requisição — POST / PUT /carros

```
{
    "marca": "Toyota", // obrigatório
    "modelo": "Corolla", // obrigatório
    "ano": 2024, // obrigatório, entre 1950 e anoAtual+1
    "placa": "ABC-1234", // obrigatório, único
    "preco": 110000.00, // obrigatório, maior que zero
    "cor": "Prata" // obrigatório
}
```

3.4 Estoque — /estoque

Método	Rota	Descrição
GET	/estoque	Lista todos os registros de estoque.
GET	/estoque/:id	Retorna um registro de estoque pelo id.
GET	/estoque/carro/:id_carro	Retorna o estoque de um carro específico.
POST	/estoque	Cria um novo registro de estoque.
PUT	/estoque/:id	Atualiza quantidade ou localização.
DELETE	/estoque/:id	Remove um registro de estoque.

Corpo da requisição — POST /estoque

```
{
    "id_carro":      1,                // obrigatório, deve existir
    "quantidade":    5,                // obrigatório, inteiro >= 0
    "localizacao_patio": "Galpão A-3", // obrigatório
    "data_entrada":   "2025-06-01"    // obrigatório, não pode ser futura
}
```

3.5 Notas Fiscais — /notas

Método	Rota	Descrição
GET	/notas	Lista todas as notas fiscais.
GET	/notas/:id	Retorna uma nota fiscal pelo id.
POST	/notas	Emite uma nova nota fiscal (decrementa estoque).

Corpo da requisição — POST /notas

```
{
    "numero_nota":    "NF-2025-0001", // obrigatório, único
    "data_emissao":   "2025-06-10",   // obrigatório, não pode ser futura
    "valor_total":    110000.00,       // obrigatório, maior que zero
    "id_cliente":     1,                // obrigatório, deve existir
    "id_vendedor":    2,                // obrigatório, deve existir
    "id_carro":       3                 // obrigatório, deve ter estoque > 0
}
```

3.6 Padrão de Respostas HTTP

A tabela a seguir descreve os códigos de status esperados em cada situação:

Status	Situação
200 OK	Requisição bem-sucedida (GET, PUT).
201 Created	Recurso criado com sucesso (POST).
400 Bad Request	Dados inválidos, campos obrigatórios ausentes ou violação de regra de negócio (ex.: CPF duplicado, carro sem estoque).
404 Not Found	Recurso não encontrado pelo id informado.
409 Conflict	Violação de unicidade (CPF, matrícula, placa, número da nota já existentes).
422 Unprocessable	Regra de negócio impede a operação (ex.: remoção de cliente com notas, estoque insuficiente).

3.7 Estrutura de Pastas Adotada

A estrutura abaixo organiza o projeto em quatro camadas bem definidas, cada uma com uma responsabilidade única:

```
projeto/
├── src/
│   ├── models/ ... Classes/tipos TypeScript (sem lógica)
│   │   ├── Cliente.ts
│   │   ├── Vendedor.ts
│   │   ├── Carro.ts
│   │   ├── Estoque.ts
│   │   └── NotaFiscal.ts
│   ├── repositories/ ... Acesso aos dados (arrays em memória)
│   │   ├── clienteRepository.ts
│   │   ├── vendedorRepository.ts
│   │   ├── carroRepository.ts
│   │   ├── estoqueRepository.ts
│   │   └── notaFiscalRepository.ts
│   ├── services/ ... Regras de negócio
│   │   ├── clienteService.ts
│   │   ├── vendedorService.ts
│   │   ├── carroService.ts
│   │   ├── estoqueService.ts
│   │   └── notaFiscalService.ts
│   ├── controllers/ ... Tratamento de Request e Response HTTP
│   │   ├── clienteController.ts
│   │   ├── vendedorController.ts
│   │   ├── carroController.ts
│   │   ├── estoqueController.ts
│   │   └── notaFiscalController.ts
│   └── app.ts ... Configuração do Express e definição das rotas
├── package.json
├── tsconfig.json
└── README.md
```

Responsabilidades de cada camada

Camada	Responsabilidade
models	Define as interfaces e tipos TypeScript que representam as entidades do domínio. Não contém nenhuma lógica.
repositories	Mantém um array privado como <i>singleton</i> (persistência em memória) e expõe funções de inserção, busca, atualização e remoção. Não valida regras de negócio.
services	Concentra toda a lógica e regras de negócio (ex.: verificar unicidade de CPF, checar estoque antes de emitir nota). Chama o <i>repository</i> para acessar os dados.

Camada	Responsabilidade
controllers	Recebe as requisições HTTP (Request), extrai os dados necessários, chama o <i>service</i> correspondente e devolve a resposta HTTP (Response) com o status adequado.
app.ts	Cria e configura a instância do Express, registra todos os <i>middlewares</i> e define as rotas, associando cada rota ao seu <i>controller</i> .

4 Entrega e Avaliação

Critérios de Avaliação

- Implementação correta da arquitetura MVC
- Cumprimento de todos os requisitos funcionais
- Aplicação rigorosa das regras de negócio
- Qualidade e organização do código
- Documentação adequada
- Desempenho na arguição

Entrega

- Data limite:** 01/06/2026
- Plataforma:** Moodle
- Submeter o **link do repositório público no GitHub** com o projeto completo contendo a *collection* do Postman com todos os testes funcionais e histórico de commits

Critérios de Avaliação

Critério	Peso	O que será observado
Organização dos commits	10%	Histórico contínuo e mensagens descritivas. Commits pontuais a cada funcionalidade implementada, sem envio único no final.
Organização do código (MVC)	20%	Separação correta entre <i>model</i> , <i>repository</i> , <i>service</i> e <i>controller</i> . Cada camada com responsabilidade única e bem definida.
Funcionalidade	25%	Todos os endpoints respondendo corretamente, status HTTP adequados e regras de negócio aplicadas. A <i>collection</i> do Postman deve cobrir todos os endpoints com ao menos um exemplo de requisição válida e um de erro.

Critério	Peso	O que será observado
Lógica de programação	20%	Clareza e coerência do código: uso correto de estruturas de repetição, condicionais, funções e tipos TypeScript.
Arguição	25%	Capacidade de explicar as decisões técnicas e responder a questionamentos sobre o próprio código.
Total	100%	

Arguição

- **Data:** 03/06/2026
- **Requisitos:**
 - Demonstração do sistema funcionando
 - Explicação das decisões técnicas
 - Resposta a questionamentos

Observações

- Ausência na arguição resultará em **nota zero**, independentemente da entrega.
- Repositórios sem histórico de commits (projeto enviado de uma só vez) terão o critério correspondente zerado.
- O código entregue deve ser **de autoria própria**. Trabalhos idênticos ou gerados integralmente por ferramentas de IA serão desconsiderados.

Observação: Ausência em qualquer etapa resultará em nota zero.

5 Formação de Grupos

Os trabalhos devem ser realizados em grupos de **até 3 integrantes**. Não serão aceitas entregas em grupos maiores.

Embora o desenvolvimento seja em grupo, **cada integrante deve ter domínio integral do projeto**. A arguição será realizada de forma **individualizada**, e o avaliador poderá questionar qualquer parte do código, independentemente de quem a implementou. A nota da arguição será atribuída **individualmente**, podendo diferir entre os membros do mesmo grupo.